صقر

SAQR — Security Automated Query & Response

منصة SOC + SOAR محلية مفتوحة المصدر بذكاء اصطناعي عربي

Version:	1.0
Date:	November 2026
Language:	Arabic + English
Platform:	Windows / WSL2 / Docker
AI Model:	Qwen2.5-3B (Ollama — Local)
Stack:	Wazuh · FastAPI · PostgreSQL · Streamlit · Telegram

SAQR = Security Automated Query & Response

يدعم العمل المحلي الكامل (Air-gapped) والعمل السحابي الاقتصادي (Cloud API)، مع توليد تقارير أمنية احترافية باللغة العربية.
الجوهرية: منصة دفاعية ذكية تجمع بين مركز عمليات مصغر (Mini-SOC) والأتمتة والاستجابة (SOAR) بنمط معماري هجين
الفكرة

1 المشكلة التي يحلها المشروع

ما يقدمه صقر	الواقع الحالي	المشكلة
استجابة تلقائية فورية دون انتظار تدخل بشري في المراحل الأولى	المحلل الأمني يراقب مئات التنبيهات يدويا	نقص كوادر الـ SOC
دعم كامل للتشغيل المحلي (Air-gapped) بنموذج لغوي محلي	الحلول السحابية تتطلب إرسال البيانات للخارج	قيود سرية البيانات
طبقة تجريد هجينة (Ollama محلي + Gemini API)	الحلول الحالية إما محلية متوسطة لا تتحمل نماذج AI ضخمة أو سحابية	محدودية موارد الأجهزة
توليد تقارير رسمية بالعربية مربوطة بأطر MITRE ATT&CK	أدوات الـ SIEM/SOAR تنتج تقارير إنجليزية	غياب التقارير بالعربية
بيئة قائمة 100% على أدوات وتقنيات مفتوحة المصدر	حلول SOAR/SIEM باهظة جدا بمئات آلاف الدولارات	تكلفة الحلول التجارية

2 المعمارية الكاملة — 5 طبقات


LAYER 1: ةيئبلا ةيضارتفالا (Windows 10 VM — Hyper-V)
Sysmon (مقارملا)
Wazuh Agent (غَلِبُملا)
Honeypot Folder
C:\test-files\ (300 فلم)
Events
Alerts (Port 1514)
LAYER 2: طخ الرصد (Docker WSL2 — wazuh-manager container)
Sysmon Events --> Wazuh Manager --> قواعد Sigma/Wazuh
1-5 قروطخ يوتسمب فَنصُم هيبنت
Webhook --> FastAPI Engine (Port 5000)
LAYER 3: مرحلا رارقلا الحتمي (FastAPI + PostgreSQL)
ةباشتملا ةهباشملا (Correlation) --> ربط الأحداث الجديدة لصت
هل الشورط متكملت؟
IF events.count > 30 AND
elapsed_seconds < 5 AND
vssadmin_event = true
ليجست T0 (مهوجهلا ةيادب) --> T1 (هيبنت لوأ) --> T2 (رارق)
ةباجتسالا رماوأ قالطإ (SOAR Actions)
[PostgreSQL] tables:
alerts | incidents | actions | evidence | timings
LAYER 4: ةنيجهلا ءاكذلا ةقبطو ةباجتسالا (SOAR + Pluggable LLM)
Action 1: Process Dump
+-> procdump.exe --> evidence/sha256_timestamp.dmp
Action 2: يروفلا يكبشلا لزعلا
+-> Wazuh Active Response --> PowerShell: Block-NetAdapter
(رفريسلاب ُالصتم Wazuh Agent لظي نكل لاصتالا عطقي)
Action 3: ليجست T3 (ةركاذلا ةطقل) + T4 (لماكلا لزعلا)

```
|  Action 4: تنبيه الذكاء الاصطناعي المزود (LLM Provider Pattern):      |
|    +-- Local Mode: Ollama (Qwen2.5-1.5B / 3B) [للوزن المفتوح]         |
|    +-- Cloud Mode: Gemini API / OpenAI         [للتطوير السريع]       |
|    Input JSON:  {host, ip, technique, evidence_hash, events[]}        |
|    Output:      تقرير أمني عربي مُنسّق مع تقنيات MITRE ATT&CK         |
+-----------------------------------------------------------------------+
          v
+-----------------------------------------------------------------------+
|  LAYER 5: الإبلاغ والتحكم (Streamlit + Telegram)                      |
|                                                                       |
|  Streamlit Dashboard (RTL - عربي):                                    |
|   +-- لوحة الحوادث المباشرة (Live SOC Board)                          |
|   +-- الجاهزية والتحقيق                                               |
|   +-- مقاييس الأداء: MTTD / MTTC / False Positives                    |
|   +-- الجدول الزمني للحادثة (Timeline T0->T4)                         |
|   +-- تقرير الذكاء الاصطناعي العربي                                   |
|                                                                       |
|  Telegram Bot:                                                        |
|   +-- إشعارات فورية عند اكتشاف: "[ALERT] جهاز WIN-01 تم عزله - MTTD: 3s"|
|   +-- أمر /status: حالة الأسطول الحالية                               |
|   +-- أمر /report [id]: استرجاع تقرير حادثة بعينها                    |
+-----------------------------------------------------------------------+
```

3 هيكل المستودع (Monorepo)

```
saqr/
+-- README.md
+-- .env.example                    # المتغيرات والمفاتيح السرية
+-- docker-compose.yml              # تكوين جميع الخدمات بأمر واحد
+-- .wslconfig                      # إعدادات WSL2 المخصصة لـ 4GB RAM
|
+-- docs/
|   +-- architecture.md             # الرسم المعماري التقني
|   +-- lab-setup.md                # دليل إعداد المعمل والشبكة
|   +-- safety-rules.md             # قواعد الأمان الصارمة للفريق
|   +-- decisions/
|       +-- ADR-001-hybrid-llm.md   # قرار معماري: النهج الهجين
|       +-- ADR-002-deterministic.md# قرار معماري: لماذا القرار النهائي ليس LLM
|
+-- lab/
|   +-- network-diagram.png         # مخطط الشبكة الافتراضية
|   +-- vms/
|   |   +-- windows10-setup.ps1     # سكربت تثبيت Sysmon + Agent تلقائياً
|   |   +-- snapshots.md            # توثيق لقطات الحالة النظيفة
|   +-- attack-engine/
|       +-- ransomware-sim/
|       |   +-- sim_ransom.py       # محاكي الفدية الآمن
|       +-- phishing/               # (السيناريو الثاني)
|       +-- ddos-sim/               # (السيناريو الثالث)
|
+-- detection/
|   +-- sysmon/
|   |   +-- sysmonconfig.xml        # إعدادات SwiftOnSecurity
|   +-- wazuh/
|   |   +-- rules/
|   |       +-- local_rules.xml     # قواعد الكشف المخصصة للفدية
|   +-- sigma/
|       +-- ransomware_rules.yml    # قواعد Sigma للتطابق
|
+-- engine/                         # القلب: FastAPI
|   +-- app/
|   |   +-- main.py                 # نقطة الدخول والتطبيق الرئيسي لـ Webhook
|   |   +-- config.py               # تحميل إعدادات البيئة
|   |   +-- models.py               # نماذج PostgreSQL (SQLAlchemy)
|   |   +-- schemas.py              # عقود البيانات الموحدة (Pydantic)
|   |   +-- rules.py                # محرك القواعد
|   |   +-- correlator.py           # خوارزمية ربط الأحداث
|   |   +-- services/
|   |       +-- soar.py             # استجابة أو تنفيذ وتسجيل
|   |       +-- evidence.py         # تجزئة وتوقيع الأدلة SHA-256
|   |       +-- metrics.py          # بيانات المقاييس MTTC و MTTD
|   |       +-- llm.py              # عقل التحقيق (Provider Pattern)
|   +-- tests/
|   +-- Dockerfile
|
```

```
|   +-- saqr/
|   |   +-- actions/
|   |   |   +-- snapshot.py            # أخذ لقطة الذاكرة والعمليات
|   |   |   +-- isolate.py             # عزل الجهاز عبر جدار الحماية
|   |   |   +-- evidence.py            # حفظ Process Dump والهاش
|   |   +-- playbooks/
|   |   |   +-- ransomware.yaml        # سيناريو الفدية التلقائي
|   |   |   +-- phishing.yaml          # سيناريو التصيد (المتقدم)
|   |   |   +-- ddos.yaml              # سيناريو هجوم الحجب (المتقدم)
|   |
|   +-- ai/
|   |   +-- providers/
|   |   |   +-- base.py                # الصنف الأساسي المجرد (Abstract Base)
|   |   |   +-- gemini_provider.py     # الاتصال السريع بـ Gemini API
|   |   |   +-- ollama_provider.py     # الاتصال المحلي بـ Ollama
|   |   +-- prompts/
|   |   |   +-- incident_analysis.txt  # صياغة التقرير الأمني الاحترافي
|   |   |   +-- mitre_mapping.txt      # ربط السلوك بتقنيات الهجوم
|   |   +-- eval/
|   |       +-- quality_metrics.py     # التقييم العددي لجودة التقرير
|   |
|   +-- dashboard/                     # واجهة التحكم: Streamlit (RTL عربي)
|   |   +-- app.py
|   |   +-- pages/
|   |   |   +-- 01_live_board.py       # لوحة المراقبة المباشرة
|   |   |   +-- 02_incidents.py        # الحوادث والتحقيق الجنائي
|   |   |   +-- 03_metrics.py          # واجهة الإحصائيات (MTTD / MTTC)
|   |   |   +-- 04_reports.py          # التقارير العربية الاحترافية
|   |   +-- Dockerfile
|   |
|   +-- bot/
|   |   +-- telegram/
|   |       +-- bot.py                 # البوت الرئيسي
|   |       +-- handlers/              # الأوامر والاستعلام
|   |
|   +-- metrics/
|       +-- collector.py               # جمع التوقيتات والأرقام
|       +-- run_experiment.py          # سكربت التجارب الآلية لـ 20
|       +-- notebooks/
|           +-- analysis.ipynb         # تقرير الجودة ورسوم
```

4 عقد البيانات الموحد (JSON Alert Schema)

JSON

```
{
  "id": "uuid-v4",
  "ts": "2026-11-03T10:15:03.120Z",
  "source": "wazuh | suricata | gateway",
  "host": "WIN-VICTIM-01",
  "ip": "10.10.10.21",
  "rule_id": "100201",
  "category": "ransomware | phishing | ddos",
  "severity": 1,
  "evidence": {
    "process": "sim_ransom.exe",
    "pid": 4821,
    "files_renamed": 312,
    "vssadmin_called": true,
    "dump_path": "evidence/WIN-01_20261103_101505.dmp",
    "dump_sha256": "a3f7c2e8b9148..."
  },
  "timings": {
    "T0": "2026-11-03T10:15:00.000Z",
    "T1": "2026-11-03T10:15:03.120Z",
    "T2": "2026-11-03T10:15:03.125Z",
    "T3": "2026-11-03T10:15:04.210Z",
    "T4": "2026-11-03T10:15:04.850Z"
  },
  "raw": {}
}
```

الأزمنة:

الرمز	المعنى	كيف يحسب
T0	بداية الهجوم الفعلي	أول حدث Sysmon يسجل تغييرا في ملفات الضحية
T1	وصول التنبيه للمنصة	طابع استلام الـ Webhook من Wazuh داخل FastAPI
T2	قرار الاستجابة	لحظة تطابق شروط القاعدة الحتمية وتفعيل الـ Playbook
T3	اكتمال تفريغ الذاكرة	بعد انتهاء procdump وحساب SHA-256 للملف
T4	اكتمال العزل الشبكي	تأكيد إغلاق منافذ الاتصال عبر جدار الحماية

المقاييس العلمية المستخلصة:

- `MTTD` (Mean Time to Detect) = T1 - T0
- `MTTC` (Mean Time to Contain) = T4 - T0

> `Files Encrypted` = عدد الملفات المشفرة قبل إتمام T4

---

5 معمارية طبقة الذكاء الاصطناعي الهجينة (Hybrid & Pluggable LLM)

[!] مهم

ويستغرق من 40 إلى 90 ثانية لإنتاج التقرير الواحد، مما يعرض النظام لخطر التجمد ونفاد الذاكرة أثناء التجارب المتزامنة.
Intel Iris` بدون ذاكرة VRAM منفصلة. تشغيل نموذج 3B أو 7B محليا على المعالج يستهلك 3-4 جيجابايت من الرام
الهندسي:** جهاز العمل يحمل معالج `Intel Core i5-1135G7` وذاكرة `16GB RAM` وبطاقة رسوميات مدمجة `Xe
**واقع الموارد

5.1 استراتيجية التصميم (Provider Pattern)

(Cloud API) والنمط المحلي (Local Ollama) بتغيير متغير بيئي واحد في ملف .env دون تعديل سطر برمجي واحد في المحرك: الذكاء الاصطناعي بنمط Provider Abstraction يفصل منطق التحليل عن مزود الخدمة. يتيح ذلك التبديل التام بين النمط السحابي بدلا من تقييد النظام بخيار واحد، صممت طبقة

PYTHON

```python
# engine/app/services/llm.py
import os
import requests
from abc import ABC, abstractmethod

class BaseLLMProvider(ABC):
    @abstractmethod
    def generate_report(self, incident_data: dict, prompt_template: str) -> str:
        pass

class GeminiAPIProvider(BaseLLMProvider):
    """النظام السحابي: سريع، فعّال، ودون استهلاك رام المعالج"""
    def __init__(self, api_key: str, model: str = "gemini-1.5-flash"):
        import google.generativeai as genai
        genai.configure(api_key=api_key)
        self.model = genai.GenerativeModel(model)

    def generate_report(self, incident_data: dict, prompt_template: str) -> str:
        prompt = prompt_template.format(incident_json=str(incident_data))
        response = self.model.generate_content(prompt)
        return response.text

class OllamaLocalProvider(BaseLLMProvider):
    """النظام المحلي: آمن 100% للشبكات المعزولة (Air-gapped)"""
    def __init__(self, base_url: str = "http://host.docker.internal:11434", model: str = "qwen2.5:1.5b"):
        self.base_url = base_url
        self.model = model

    def generate_report(self, incident_data: dict, prompt_template: str) -> str:
        prompt = prompt_template.format(incident_json=str(incident_data))
        res = requests.post(
            f"{self.base_url}/api/generate",
            json={"model": self.model, "prompt": prompt, "stream": False},
            timeout=180
        )
        return res.json().get("response", "")

def get_llm_provider() -> BaseLLMProvider:
    """مصنع الموزّد بحسب إعدادات البيئة"""
    provider_type = os.getenv("LLM_PROVIDER", "gemini").lower()
    if provider_type == "gemini":
        return GeminiAPIProvider(api_key=os.getenv("GEMINI_API_KEY", ""))
    elif provider_type == "ollama":
        model_name = os.getenv("OLLAMA_MODEL", "qwen2.5:1.5b")
        return OllamaLocalProvider(model=model_name)
    raise ValueError(f"Unknown provider: {provider_type}")
```

5.2 الإعداد والتحكم عبر متغيرات البيئة (.env)

BASH

```
# للتبديل الفوري دون لمس الكود:

# النمط الأول: سحابي فائق السرعة والخفة (موصى به أثناء التطوير والتجارب الـ 20)
LLM_PROVIDER=gemini
GEMINI_API_KEY=AIzaSyD...
GEMINI_MODEL=gemini-1.5-flash

# النمط الثاني: محلي كامل للبيئات المعزولة (يوم العرض أو البيئات الصارمة)
# LLM_PROVIDER=ollama
# OLLAMA_URL=http://host.docker.internal:11434
# OLLAMA_MODEL=qwen2.5:1.5b
```

5.3 خطة التنفيذ المرحلية للذكاء الاصطناعي

1. مرحلة التطوير والاختبارات الـ 20 (الأسابيع 1 إلى 10): >

تفعيل `LLM_PROVIDER=gemini` عبر واجهة مجانية. >

المكسب: توفير 3.5GB رام بالكامل لمحرك Wazuh والآلة الافتراضية، وسرعة استجابة فائقة (< 3 ثوان للتقرير)، مع تجنب انهيار النظام أثناء تكرار الت

2. مرحلة تجهيز العرض والتقييم المحلي (الأسابيع 11 إلى 13): >

تثبيت `Ollama` وتجربة نموذج `qwen2.5:1.5b` (حجمه 1 جيجابايت فقط، يحتاج 1.5GB رام فقط على الـ CPU، وزمنه ~20 ثانية). >

إجراء اختبار مقارنة بين التقرير المحلي والسحابي لتوثيقه في فصل المناقشة. >

3. يوم العرض والمناقشة: >

إن كان الجهاز مستقرا والموارد كافية: يعرض بنمط `ollama` لإبراز ميزة العمل بلا إنترنت. >

إن ظهر أي بطء أو ضغط غير متوقع: يحول في ثوان إلى `gemini` بضمان استقرار العرض 100%. >

5.4 الرواية الهندسية والأكاديمية أمام لجنة التحكيم

مناقشة المشروع أمام اللجنة، لا يقدم الأمر كـ "عجز في الأجهزة"، بل كـ ميزة تصميم معماري متقدمة (Enterprise Flexibility):

عند

Cloud عبر Gemini API) لتوفير تكلفة العتاد وتسريع استخراج التقارير. النظام يدعم الخيارين بسلاسة عبر سطر تكوين واحد."*

و Qwen2.5) لضمان عدم خروج أي بايت خارج الشبكة. بينما الفروع والمؤسسات المتوسطة تستطيع اختيار النمط السحابي (Mode

فالمؤسسات الدفاعية والسيادية التي تطبق عزلا فيزيائيا صارما (Air-gapped) تشغل صقر بالنمط المحلي (Local Mode عبر Ollama

*"حرصنا في صقر على اتباع معمارية المؤسسات المرنة (Pluggable Architecture)؛

---

6 سيناريو الفدية الكامل خطوة بخطوة

6.1 قبل تشغيل الهجوم (الحالة النظيفة)

```
[OK] آلة افتراضية Windows 10 تعمل وظوفحم ا Clean Snapshot
[OK] أدوات المراقبة Sysmon و Wazuh Agent تسجل وترسل الحوادث
[OK] حاويات Docker Compose تعمل (wazuh-manager, engine, postgres, dashboard, bot)
[OK] مجلد Honeypot التجريبي C:\test-files\ يحتوي 300 ملف وهمي
[OK] مزامنة الوقت عبر NTP شغالة ونظام المراقبة يرى الآلة الافتراضية
```

6.2 تشغيل الهجوم الآمن

PYTHON

```python
# lab/attack-engine/ransomware-sim/sim_ransom.py
import os, time, glob

HONEYPOT = r"C:\test-files"
START_TIME = time.time()  # تسجيل T0 للقياس

files = glob.glob(f"{HONEYPOT}\\*")
for f in files:
    # محاكاة سلوك البرمجية الخبيثة: قراءة وتشفير وحذف الأصل بسرعة عالية
    os.rename(f, f + ".saqr_locked")
    time.sleep(0.01)

# محاكاة أمر حذف النسخ الاحتياطية الشهير
os.system("vssadmin delete shadows /all /quiet")

print(f"T0: {START_TIME} | Processed: {len(files)}")
```

6.3 التسلسل الزمني للاستجابة التلقائية

```
t=0.000s | sim_ransom.py يدبي أ ةعلم ا ل يدعتو ل مل ا تافل
         |
t=0.010s | Sysmon ري دصر ا ل ثادحأ ا ل: EventID 11 (File Create/Rename) و EventID 1 (vssadmin)
         |
t=0.100s | Wazuh Agent ري سر ل ا ثادحأ ا ل ا وف ُ ار ل ل ديم ر يبع ر بع م ن ذف 1514
         |
t=0.500s | Wazuh Manager قبطي ا ل وقو ا ع د:
         |   Rule 100201: ع ا م ج ت غ ي ي ر ف ش ك ف ل م 30 <) ن ِ ا و ث 5 ل ا ل خ
         |   Rule 100202: ك ش ف م ح ا و ل ة ذ ف ا ظ ل vssadmin
         |
t=0.700s | Wazuh ري سر ل Webhook وف ر ي إ ل ى ك ر ح FastAPI (/alerts) --> ت س ج ي ل T1
         |
t=0.705s | ك ر ح ا ل ق ر ا ر ا ل ح ت م ي (FastAPI Correlator):
         |   ة ط ب ا ق ة ر ش و ر ط ا ل ف د ي ة ل م ؤ ك د ة --> ت س ج ي ل T2 (ق ر ا ر ا ل ا ح ت و ا ء)
         |
t=0.710s | إ ل ط ا ق أ و ا ر م ا ل ا ت س ا ج ا ب ة ا ل م ت ا ز م ن ة (SOAR Actions):
         |   1. أ خ ذ Process Dump ل ل ع ل م ي ة و ح س ا ب SHA-256 --> ت س ج ي ل T3
         |   2. ع ز ل ا ل ج ا ه ز ا ش ب ك ي ا ُ ب ق ط ع ك ر ح ة ا ل م ر و ر ب ع Windows Firewall --> ت س ج ي ل T4
         |   3. إ ر س ا ل ا ش إ ع ا ر ا ع ل ج ا ل ع ب ر ت و ب Telegram ل ل د ا ر إ ة
         |
t=1.000s | ا س ت د ا ع ء ط ب ق ة ق ب ط ء ا ل ذ ك ء ا ل ا ص ط ن ا ع ي (LLM Provider):
         |   إ ر س ا ل ب ي ا ن ا ت ا ل ا ح د ا ث ة ا ل م ج م ع ة ء ا ش ن إ ل ا ت ا ر ي ق ت ر ي ا ل ع ب ي
         |   (ق ر غ ت س ي < 3 ث ا و ن ِ ف ي ط م ن Gemini و أ ~20 ة ي ن ا ث ف ي ط م ن Ollama 1.5B)
         |
t=3.500s | ة ج و ل Streamlit ت ث د ح ت ت ل ق ا ل ا ئ ي ا ُ:
         |   ا ل ح ا ل ة: [!] ل و ز و م ع ز ا ه ج | MTTD: 0.70s | MTTC: 1.15s | ا ل ت ق ر ي ر ا ل ع ر ب ي ز ا ه ج
```

7 بروتوكول التجارب الـ 20 (Evaluation Protocol)

7.1 منهجية القياس العلمي

لضمان الدقة الأكاديمية والمصداقية الإحصائية أمام لجان التحكيم، ينفذ سيناريو الهجوم 20 مرة متتالية ببروتوكول موحد لحساب:

- متوسط زمن الكشف (Mean MTTD) والانحراف المعياري.
- متوسط زمن الاحتواء (Mean MTTC) والانحراف المعياري.
- نسبة الإنذارات الكاذبة (False Positive Rate) عبر تشغيل تطبيقات اعتيادية ضخمة (نسخ احتياطي شرعي، ضغط ملفات).
- مقارنة Red vs Blue: توثيق الفرق الصادم بين غياب المنصة (تشفير 100% من الملفات بعد دقائق) ووجود المنصة (توقف الهجوم في أقل من ثانيتين

7.2 سكربت تشغيل التجارب الآلية

PYTHON

```
# metrics/run_experiment.py
import subprocess, time, csv, requests, statistics
from datetime import datetime

RUNS = 20
RESULTS = []

def run_test(run_id):
    print(f"\n> بدء التجربة رقم {run_id}...")
    input("  تأكد من استعادة الـ Snapshot النظيفة واضغط Enter...")
    time.sleep(3)

    # تشغيل محاكي الفدية على الضحية
    subprocess.Popen([
        "powershell.exe", "-Command",
        "Invoke-Command -VMName 'SAQR-Win10' -ScriptBlock { python C:\\lab\\sim_ransom.py }"
    ])

    # انتظار اكتمال الاستجابة
    time.sleep(15)

    # جلب التوقيتات المسجلة من المحرك
    data = requests.get("http://localhost:5000/metrics/latest").json()
    RESULTS.append(data)
    print(f"  [OK] تجربة {run_id}: MTTD={data['mttd_ms']}ms, MTTC={data['mttc_ms']}ms")

for i in range(1, RUNS + 1):
    run_test(i)

# حفظ النتائج في ملف CSV للأوراق البحثية
with open("saqr_experiment_results.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=RESULTS[0].keys())
    writer.writeheader()
    writer.writerows(RESULTS)

mttds = [r["mttd_ms"] for r in RESULTS]
mttcs = [r["mttc_ms"] for r in RESULTS]
print("\n" + "="*50)
print(f"متوسط زمن الكشف (MTTD): {statistics.mean(mttds):.1f} ms (انحراف: {statistics.stdev(mttds):.1f})")
print(f"متوسط زمن الاحتواء (MTTC): {statistics.mean(mttcs):.1f} ms (انحراف: {statistics.stdev(mttcs):.1f})")
```

8 إعداد البيئة خطوة بخطوة (Lab Setup)

8.1 ميزانية موارد الجهاز

```
المعالج:  Intel Core i5-1135G7 (4 أنوية / 8 خيوط)
الذاكرة:   16GB RAM:
          +-- 4.0GB مخصصة لـ Docker / WSL2
          +-- 3.5GB مخصصة لآلة افتراضية Windows 10
          +-- 1.5GB مخصصة لـ Ollama (عند تفعيل النطاق المحلي)
          +-- 7.0GB للنظام والتشغيل وبقية البرامج
المساحة:  متاحة إلى ~60GB من أصل 90GB موفرة
```

8.2 ملف .wslconfig (إلزامي في دليل المستخدم)

INI

```
# C:\Users\<Username>\.wslconfig
[wsl2]
memory=4GB
processors=4
swap=2GB
localhostForwarding=true
networkingMode=mirrored
```

8.3 ملف Docker Compose

YAML

```yaml
# deploy/docker-compose.yml
version: "3.9"

services:
  wazuh-manager:
    image: wazuh/wazuh-manager:4.7.3
    container_name: saqr-wazuh
    ports:
      - "1514:1514/udp"
      - "1515:1515"
      - "55000:55000"
    volumes:
      - wazuh-data:/var/ossec/data
      - ./detection/wazuh/rules:/var/ossec/etc/rules/local:ro
    environment:
      - WAZUH_MANAGER_IP=0.0.0.0
    networks:
      - saqr-net

  postgres:
    image: postgres:16-alpine
    container_name: saqr-db
    environment:
      POSTGRES_DB: saqr
      POSTGRES_USER: saqr
      POSTGRES_PASSWORD: ${DB_PASSWORD:-saqr123}
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
      - saqr-net

  engine:
    build: ../engine
    container_name: saqr-engine
    ports:
      - "5000:5000"
    environment:
      - DB_URL=postgresql://saqr:saqr123@postgres/saqr
      - WAZUH_URL=http://wazuh-manager:55000
      - LLM_PROVIDER=${LLM_PROVIDER:-gemini}
      - GEMINI_API_KEY=${GEMINI_API_KEY}
      - OLLAMA_URL=http://host.docker.internal:11434
      - TELEGRAM_TOKEN=${TELEGRAM_TOKEN}
      - TELEGRAM_CHAT_ID=${TELEGRAM_CHAT_ID}
    depends_on:
      - postgres
      - wazuh-manager
    networks:
      - saqr-net
```

```
  dashboard:
    build: ../dashboard
    container_name: saqr-dashboard
    ports:
      - "8501:8501"
    environment:
      - ENGINE_URL=http://engine:5000
    depends_on:
      - engine
    networks:
      - saqr-net

volumes:
  wazuh-data:
  postgres-data:

networks:
  saqr-net:
    internal: false
```

9 قواعد الكشف الرئيسية (Detection Rules)

XML

```
<!-- detection/wazuh/rules/local_rules.xml -->
<group name="saqr,ransomware,">

  <!-- كشف التشفير الجماعي السريع -->
  <rule id="100201" level="14" frequency="30" timeframe="5">
    <if_matched_sid>60106</if_matched_sid>
    <description>SAQR: Rapid Mass File Modifications - Ransomware Indicator</description>
    <group>ransomware_indicator,</group>
  </rule>

  <!-- كشف محاولة حذف النسخ الاحتياطية -->
  <rule id="100202" level="15">
    <if_sid>61613</if_sid>
    <field name="win.eventdata.commandLine" type="pcre2">(?i)vssadmin.*delete.*shadows</field>
    <description>SAQR: Shadow Copies Deletion Attempt - Critical Impact</description>
    <group>ransomware_indicator,</group>
  </rule>

  <!-- الحادثة المركبة: إصدار أمر الاجتياح الفوري -->
  <rule id="100210" level="15">
    <if_matched_group>ransomware_indicator</if_matched_group>
    <description>SAQR: High-Confidence Ransomware Outbreak - Trigger Automated SOAR</description>
  </rule>

</group>
```

10 صياغة التقرير الأمني باللغة العربية (LLM Prompting)

PYTHON

```
# ai/prompts/incident_analysis.txt

PROMPT_TEMPLATE = """أنت محلل استجابة للحوادث السيبرانية في مركز عمليات الأمن (SOC).
قيّم بيانات الحادثة الأمنية المحتواة آليًا ُ التالية بصيغة JSON:

{incident_json}

المطلوب صياغة تقرير رسمي متكامل باللغة العربية وفق الأقسام التالية:

1. **الملخص التنفيذي:**
   شرح موجز وغير تقني موجه إلدارة المؤسسة يلخص ما حدث، والسمة البيانات، ونجاح العزل التلقائي.

2. **التسلسل الزمني للحادثة (Timeline):**
   جدول زمني يوضح أوقات الهجوم (T0)، الكشف (T1)، اتخاذ القرار (T2)، حفظ الأدلة (T3)، والعزل التام (T4).

3. **مطابقة إطار MITRE ATT&CK:**
   ذكر التقنيات المرصودة (مثل T1486 للتشفير، T1490 لتعطيل استعادة النظام).

4. **الأدلة الجنائية المحفوظة:**
   اسم العملية المشبوهة، معرّفها (PID)، مسار ملف الذاكرة (Process Dump)، والتجزئة المشفرة SHA-256 لضمان حجية ا

5. **الإجراءات التلقائية المتخذة:**
   تفاصيل تفعيل جدار الحماية وعزل الجهاز في زمن قياسي.

6. **التوصيات العاجلة والوقائية:**
   خطوات مسح النظام، استعادة الملفات المتضررة، وتأمين بقية الشبكة.
"""
```

11 مقاييس تقييم جودة التقرير العربي (LLM Evaluation)

PYTHON

```
# ai/eval/quality_metrics.py

EVALUATION_METRICS = {
    "completeness": {
        "weight": 0.25,
        "criteria": "الاشتمال التقرير على الملخص، التايم لاين، والأدلة الجنائية"
    },
    "mitre_fidelity": {
        "weight": 0.25,
        "criteria": "صحة أرقام ومسميات تقنيات MITRE ATT&CK المذكورة"
    },
    "technical_consistency": {
        "weight": 0.30,
        "criteria": "تطابق قيم الهاش والأوقات والأرقام بدقة مع بيانات JSON المدخلة"
    },
    "arabic_clarity": {
        "weight": 0.20,
        "criteria": "السلامة التعبيرية الأمنية العربية وخلوه من الركاكة أو الترجمة الحرفية"
    }
}
# التقييم الإجمالي = مجموع حاصل ضرب الدرجات بالأوزان (الهدف المعتمد > 8.0 / 10)
```

12 لوحة التحكم التفاعلية (Streamlit RTL)

PYTHON

```
# dashboard/app.py
import streamlit as st
import requests

st.set_page_config(page_title="صَقر | منصة الاستجابة الآلية", layout="wide", page_icon="[SAQR]")
st.markdown('<style>body { direction: rtl; text-align: right; }</style>', unsafe_allow_html=True)

st.title("[SAQR] منصة صَقر — مركز العمليات المصغر (Mini-SOC)")

col1, col2, col3, col4 = st.columns(4)
col1.metric("حالة المنظومة", "نشطة ")
col2.metric("متوسط زمن الكشف (MTTD)", "0.72 ثانية", delta="-0.15s")
col3.metric("متوسط زمن الاحتواء (MTTC)", "1.18 ثانية", delta="آلي [!]")
col4.metric("الأجهزة المحتجزة", "1 زاجر معزول ")

st.markdown("---")
st.subheader("[ALERT] سجل الحوادث المباشر")
# عرض التقارير العربية وتاريخ الاختراقات لإضافة المزيد
```

13

YAML

```yaml
# soar/playbooks/ransomware.yaml
playbook_name: "Ransomware Automated Containment"
version: "2.0"
trigger_rules: [100201, 100202, 100210]

execution_flow:
  - step: 1
    action: "engine.record_timestamp"
    target: "T2"

  - step: 2
    action: "agent.capture_process_dump"
    params:
      tool: "procdump.exe"
      hash_algorithm: "SHA-256"
    record_metric: "T3"
    timeout_sec: 15

  - step: 3
    action: "agent.network_quarantine"
    params:
      method: "windows_firewall_rule"
      whitelist_ip: "wazuh_server_ip"
    record_metric: "T4"
    timeout_sec: 5

  - step: 4
    action: "notifier.telegram_broadcast"
    params:
      priority: "CRITICAL"

  - step: 5
    action: "ai.generate_arabic_report"
    mode: "async"
```

14 نطاق السيناريوهات للمشروع (سيمستر 1 و 2)

السيناريو	السيمستر	الهدف العملي	آلية الاستجابة التلقائية
[!] **برامج الفدية (Ransomware)	الأول (الأساسي)	عزل جدار الحماية + تقرير عربي + Process Dump + تشفير البيانات في ثوان	
[~] **التصيد الاحتيالي (Phishing)		حظر النطاق بالجدار الناري + إلغاء الجلسات والنطاقات المزورة	الثاني
[~] **حجب الخدمة الموزع (DDoS)		المهاجمة IP ديناميكي + حظر عناوين الهجوم عن طريق Rate Limiting	الطبيعي

توزيع المهام على فريق العمل (5 طلاب) 15

الطالب	المسمى والمسؤولية	المهام التنفيذية المحددة	مقياس الإنجاز
الطالب 1		تجهيز Win10 VM، وضبط محاكي الهدية، والعمل على Snapshots	ثبات سيناريو الهجوم في 20 تشغيلة متتالية
الطالب 2		تثبيت وإعداد Wazuh Manager و Sysmon، وصياغة قواعد كشف التهديدات	رصد الهجوم بدقة > 95% وزمن أقل من 10 ثوان
الطالب 3		تطوير واجهات FastAPI ومحرك القواعد وتخزين الحوادث والقرار	زمن معالجة الأحداث والقرار أقل من 5 ثوان
الطالب 4		تطبيق LLMProvider، وإدارة طبقة الذاكرة والذكاء الاصطناعي	عزل تام في أقل من 5 ثوان وتقارير عربية
الطالب 5		بناء لوحة التحكم Streamlit، وبوت Telegram، وتوثيق النتائج	جداول إحصائية ورسوم بيانية دقيقة لكتابة الفصل

الجدول الزمني التنفيذي (15 أسبوعا) 16

الأسبوع	التركيز الهندسي	المخرجات المستهدفة (Milestones)
الأسبوع 1		ضبط `wslconfig.`، وتشغيل Docker Compose لبنية تحتية VM البيئة الصحية
الأسبوع 2 - 3		تفعيل Sysmon و Wazuh Agent، والتحقق من ظهور الأحداث الأولية
الأسبوع 4 - 5		كتابة `sim_ransom.py` الآمن، واعتماد قواعد Wazuh الفكية وقواعد الكشف
الأسبوع 6 - 7		ربط Webhook بـ FastAPI، وتنفيذ سكربت العزل واختبار الأوامر المحاكاة
الأسبوع 8 - 9		تطبيق `LLMProvider` (الربط بـ Gemini API وبطاقة الذاكرة والتحليل الذكي) الهجينة
الأسبوع 10 - 11		إتمام لوحة Streamlit العربية، وتفعيل إشعارات التنبيه والتحكم وبوت Telegram
الأسبوع 12		اختبار الدورة الكاملة من نقرة الهجوم إلى صدور التقرير النهائي والتنقيح
الأسبوع 13		تشغيل بروتوكول القياس وحساب متوسطات MTTC و MTTD والـ 20 الحماية
الأسبوع 14		توثيق النتائج بالرسوم البيانية، وتسجيل فيديو تجريبي احتياطي (Demo)
الأسبوع 15		محاكاة المناقشة أمام اللجنة والتأكد من جاهزية نسخة الـ VM العرض النهائي

المخرجات المسلمة للجنة التحكيم 17

معيار التقييم والقبول	شكل التسليم	المخرج
إمكانية تشغيل المنظومة بسهولة عبر سكريبت موحدموثق Git Repository		مستودع الكود المنظم
اكتمال حلقة: هجوم --> كشف --> عزل --> تقرير	تشغيل فعلي أمام اللجنة	عرض حي متكامل (Live Demo)
توثيق 20 تشغيلة قياسية بمتوسط MTTD < 2s و	جداول CSV ورسوم بيانية	بيانات التجارب الإحصائية
توثيق المعمارية الهجينة، والمنهجية العلمية، وتحليل	أكاديمي رسمي	كتيب تقرير التخرج
ضمان تقديم المشروع بسلاسة حتى لو حدث أي	في الدقة (10 دقائق)	فيديو العرض الاحتياطي

[!] مهم

الـ LLM، دور الذكاء الاصطناعي هو التحليل وصياغة التقارير **بعد اكتمال الاحتواء**، مما يضمن أمان وسرعة المنظومة. للمشروع:** قرارات العزل وحفظ الأدلة تتخذ فوريا وبشكل **حتمي 100% (Deterministic Engine)** دون انتظار **القاعدة الحاكمة